

# Documentazione Assembler per Esercitazioni di Reti Logiche A.A. 2024/25

Raffaele Zippo

4 giugno 2026

# Indice

|          |                                         |           |
|----------|-----------------------------------------|-----------|
| <b>1</b> | <b>Architettura x86</b>                 | <b>3</b>  |
| 1.1      | Registri                                | 3         |
| 1.2      | Memoria                                 | 4         |
| 1.3      | Spazio di I/O                           | 4         |
| 1.4      | Condizioni al reset                     | 4         |
| <b>2</b> | <b>Istruzioni processore x86</b>        | <b>5</b>  |
| 2.1      | Spostamento di dati                     | 5         |
| 2.2      | Aritmetica                              | 5         |
| 2.3      | Logica binaria                          | 6         |
| 2.4      | Traslazione e Rotazione                 | 6         |
| 2.5      | Controllo di flusso                     | 7         |
| 2.6      | Operazioni condizionali                 | 8         |
| 2.7      | Istruzioni stringa                      | 9         |
| 2.8      | Altre istruzioni                        | 10        |
| <b>3</b> | <b>Sottoprogrammi di utility</b>        | <b>11</b> |
| 3.1      | Terminologia                            | 11        |
| 3.2      | Caratteri speciali                      | 11        |
| 3.3      | Sottoprogrammi                          | 11        |
| <b>4</b> | <b>Debugger gdb</b>                     | <b>13</b> |
| 4.1      | Controllo dell'esecuzione               | 13        |
| 4.2      | Ispezione dei registri                  | 14        |
| 4.3      | Ispezione della memoria                 | 14        |
| 4.4      | Gestione dei breakpoints                | 15        |
| <b>5</b> | <b>Tabella ASCII</b>                    | <b>17</b> |
| <b>6</b> | <b>Ambiente d'esame e i suoi script</b> | <b>19</b> |
| 6.1      | Aprire l'ambiente                       | 19        |
| 6.2      | Il terminale Powershell                 | 20        |
| 6.3      | Eseguire gli script                     | 20        |
| <b>7</b> | <b>Problemi comuni</b>                  | <b>23</b> |
| 7.1      | Setup dell'ambiente                     | 23        |
| 7.2      | Uso dell'ambiente                       | 24        |
| <b>8</b> | <b>Essere efficienti con VS Code</b>    | <b>25</b> |
| 8.1      | Le basi elementari                      | 25        |
| 8.2      | Le basi un po' meno elementari          | 25        |
| 8.3      | Editing multi-caret                     | 25        |

# 1. Architettura x86

Riportiamo qui una vista *semplificata e riassuntiva* dell'architettura x86 per la quale scriveremo programmi assembler.

L'architettura x86 è a 32 bit. Questo implica che i registri generali, così come tutti gli indirizzi per locazioni in memoria, sono a 32 bit. L'evoluzione di questa architettura, x64 a 64 bit, che è quella che troviamo nei processori in commercio, è del tutto retrocompatibile.

## Importanti semplificazioni

La visione del processore che proponiamo è molto limitata, e omette diversi importanti registri, flag e funzionalità che saranno esplorati in corsi successivi. Questi includono, per esempio, il registro `ebp`, la natura dei meccanismi di protezione, il significato di `SEGMENTATION FAULT`, e che cosa sia un *kernel*.

Quanto discutiamo è tuttavia sufficiente agli scopi didattici di questo corso.

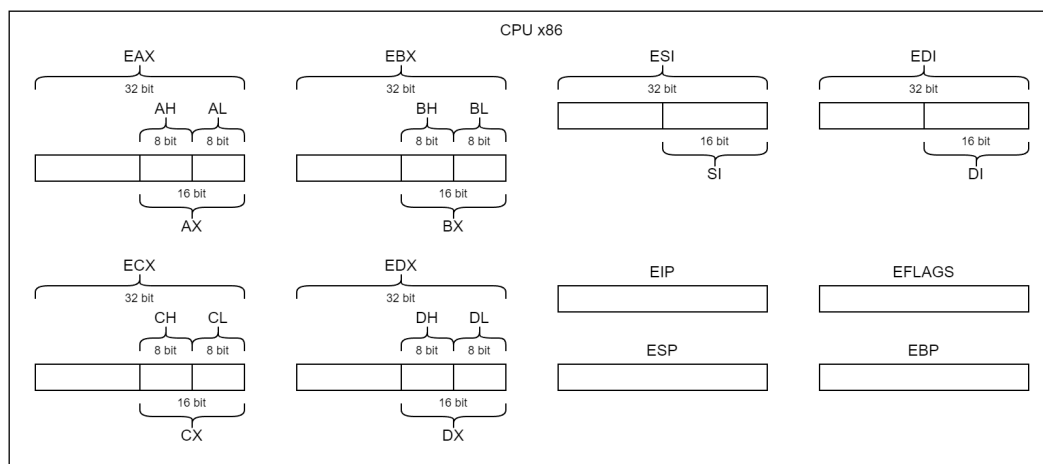
## 1.1 Registri

I registri che utilizzeremo *direttamente* sono 6: `eax`, `ebx`, `ecx`, `edx`, `esi`, `edi`. Per i primi quattro di questi, è possibile operare sulle loro porzioni a 16 e 8 bit tramite `ax`, `ah`, `al` e così via. Per i registri `esi` ed `edi` è possibile operare solo sulle porzioni a 16 bit, tramite `si` e `di`. Tipicamente, i registri `eax...` `edx` sono utilizzati per processare dati, mentre `esi` ed `edi` sono utilizzati come registri puntatori. Questa divisione di utilizzo non è però affatto obbligatoria per la maggior parte delle istruzioni.

Altri registri sono invece utilizzati in modo indiretto:

- `esp` è il registro puntatore per la *cima* dello stack, viene utilizzato da `pop` / `push` per prelevare/spostare valori nella pila, e da `call` / `ret` per la chiamata di sottoprogrammi;
- `eip` è il registro puntatore verso la prossima istruzione da eseguire, viene incrementato alla fine del *fetch* di una istruzione e modificato da istruzioni che cambiano il flusso d'esecuzione, come `call`, `ret` e le varie `jmp`;
- `eflags` è il registro dei flag, una serie di booleani con informazioni sullo stato dell'esecuzione e sul risultato dell'ultima operazione aritmetica. I flag di nostro interesse sono il carry flag `CF` (posizione 0), lo zero flag `ZF` (6), il sign flag `SF` (7), l'overflow flag `OF` (11). Sono tipicamente aggiornati dalle istruzioni aritmetiche, e testati indirettamente con istruzioni condizionali come `jcon`, `set` e `cmov`.

Di seguito uno schema funzionale dei registri del processore x86.



## 1.2 Memoria

Lo spazio di memoria dell'architettura x86 è indirizzato su 32 bit. Ciascun indirizzo corrisponde a un byte, ma è possibile eseguire anche letture e scritture a 16 e 32 bit.

Per tali casi è importante ricordare che l'architettura x86 è *little-endian*, che significa **little end first**, [un riferimento a I viaggi di Gulliver](#). Questo si traduce nel fatto che quando un valore di  $n$  byte viene salvato in memoria *a partire* dall'indirizzo  $a$ , il byte meno significativo del valore viene salvato in  $a$ , il secondo meno significativo in  $a + 1$ , e così via fino al più significativo in  $a + (n - 1)$ .

Questo ordinamento dei bytes in memoria non inficia sulla coerenza dei dati nei registri: eseguendo `movl %eax, a` e `movl a, %eax` il contenuto di `eax` non cambia, e l'ordinamento dei bit rimane coerente.

I *meccanismi di protezione* ci precludono l'accesso alla maggior parte dello spazio di memoria. Potremmo accedere senza incorrere in errori solo

1. allo stack
2. allo spazio allocato nella sezione `.data`
3. alle istruzioni nella sezione `.text`

Queste sezioni tipicamente non includono gli indirizzi “bassi”, cioè a partire da `0x0`.

È importante anche tenere presente che

1. non è possibile *eseguire* istruzioni dallo stack e da `.data`
2. non è possibile *scrivere* nella sezione `.text`

Vanno quindi opportunamente dichiarate le sezioni, e vanno evitate operazioni di `jmp`, `call` etc. verso locazioni di `.data` così come le `mov` verso locazioni di `.text`.

In caso di violazione di questi meccanismi, l'errore più tipico è `SEGMENTATION FAULT`.

## 1.3 Spazio di I/O

Lo spazio di I/O, sia quello fisico (monitor, speaker, tastiera, etc.) sia quello virtuale (terminale, files su disco, etc.) ci è in realtà precluso tramite *meccanismi di protezione*. Tentare di eseguire istruzioni `in` o `out` porterà infatti al brusco arresto del programma. Il nostro programma può interagire con lo spazio di I/O solo tramite il *kernel* del *sistema operativo*.

Tutta questa complessità è astratta tramite i *sottoprogrammi di input/output* dell'ambiente, documentati [qui](#).

## 1.4 Condizioni al reset

Il reset iniziale e l'avvio del nostro programma sono concetti completamente diversi e scollegati. Non possiamo sfruttare nessuna ipotesi sullo stato dei registri al momento dell'avvio del nostro programma, se non che il registro `eip` punterà a un certo punto alla prima istruzione di `_main`.

Il fatto che `_main` sia l'entry point del nostro programma, così come l'uso di `ret` senza alcun valore di ritorno, è una caratteristica di *questo ambiente*.

## 2. Istruzioni processore x86

Le seguenti tabelle sono per *riferimento rapido* : sono utili per la programmazione pratica, ma omettono molteplici dettagli che serve sapere, e che trovate nel resto del materiale.

Si ricorda che utilizziamo la sintassi GAS/AT&T, dove le istruzioni sono nel formato *opcode source destination*. Nella colonna notazione, indicheremo con [bwl] le istruzioni che richiedono la specifica delle dimensioni. Quando la dimensione è deducibile dai registri utilizzati, questi suffissi si possono omettere. Per gli operandi, useremo le seguenti sigle:

- *r* per un registro (come in `mov %eax, %ebx`);
- *m* per un indirizzo di memoria;
- *i* per un valore immediato (come in `mov $0, %eax`).

Per gli indirizzi in memoria, abbiamo a disposizione tre notazioni:

- immediato, come in `mov numero, %eax`;
- tramite registro, come in `mov (%esi), %eax`;
- con indice, come in `mov matrice(%esi, %ecx, 4), %eax`.

Si ricorda che non tutte le combinazioni sono permesse nell'architettura x86: nessuna istruzione generale supporta l'indicazione di *entrambi* gli operandi in memoria (cioè, non si può scrivere `movl x, y` o `mov (%eax), (%ebx)`). Fanno eccezione le istruzioni stringa come la **movs**, usando operandi impliciti.

### 2.1 Spostamento di dati

| Istruzione | Nome esteso                | Notazione           | Comportamento                                                                  |
|------------|----------------------------|---------------------|--------------------------------------------------------------------------------|
| mov        | Move                       | mov[bwl] r/m/i, r/m | Scrive il valore sorgente nel destinatario. Non modifica alcun flag.           |
| lea        | Load Effective Address     | lea m, r            | Scrive l'indirizzo m nel registro destinatario.                                |
| xchg       | Exchange                   | xchg[bwl] r/m, r/m  | Scambia il valore del sorgente con quello del destinatario.                    |
| cbw        | Convert Byte to Word       | cbw                 | Estende il contenuto di %al su %ax, interpretandone il contenuto come intero.  |
| cwde       | Convert Word to Doubleword | cwde                | Estende il contenuto di %ax su %eax, interpretandone il contenuto come intero. |
| push       | Push onto the Stack        | push[wl] r/m/i      | Aggiunge il valore sorgente in cima allo stack (destinatario implicito).       |
| pop        | Pop from the Stack         | pop[wl] r/m         | Rimuove un valore dallo stack (sorgente implicito) lo scrive nel destinatario. |

### 2.2 Aritmetica

| Istruzione | Nome esteso             | Notazione           | Comportamento                                                                                                                                  |
|------------|-------------------------|---------------------|------------------------------------------------------------------------------------------------------------------------------------------------|
| add        | Addition                | add[bwl] r/m/i, r/m | Somma sorgente e destinatario, scrive il risultato sul destinatario. Valido sia per naturali che interi. Aggiorna SF, ZF, CF e OF.             |
| sub        | Subtraction             | sub[bwl] r/m/i, r/m | Sottrae il sorgente dal destinatario, scrive il risultato sul destinatario. Valido sia per naturali che interi. Aggiorna SF, ZF, CF e OF.      |
| adc        | Addition with Carry     | adc[bwl] r/m/i, r/m | Somma sorgente, destinatario e CF, scrive il risultato sul destinatario. Valido sia per naturali che interi. Aggiorna SF, ZF, CF e OF.         |
| sbb        | Subtraction with Borrow | sbb[bwl] r/m/i, r/m | Sottrae il sorgente e CF dal destinatario, scrive il risultato sul destinatario. Valido sia per naturali che interi. Aggiorna SF, ZF, CF e OF. |
| inc        | Increment               | inc[bwl] r/m        | Somma 1 (sorgente implicito) al destinatario. Aggiorna SF, ZF, e OF, ma non CF.                                                                |
| dec        | Decrement               | dec[bwl] r/m        | Sottrae 1 (sorgente implicito) al destinatario. Aggiorna SF, ZF, e OF, ma non CF.                                                              |
| neg        | Negation                | neg[bwl] r/m        | Sostituisce il destinatario con il suo opposto. Aggiorna ZF, SF e OF. Modifica CF.                                                             |

Le seguenti istruzioni hanno operandi e destinatari impliciti, che variano in base alla dimensione dell'operazione. Usano inoltre composizioni di più registri: useremo `%dx_%ax` per indicare un valore i cui bit più significativi sono scritti in `%dx` e quelli meno significativi in `%ax`.

| Istruzione | Nome esteso               | Notazione | Comportamento                                                                                                                                                                   |
|------------|---------------------------|-----------|---------------------------------------------------------------------------------------------------------------------------------------------------------------------------------|
| mul        | Unsigned Multiply, 8 bit  | mulb r/m  | Calcola su 16 bit il prodotto tra naturali del sorgente e %al, scrive il risultato su %ax. Se il risultato non è riducibile a 8 bit, mette CF e OF a 1, altrimenti a 0.         |
| mul        | Unsigned Multiply, 16 bit | mulw r/m  | Calcola su 32 bit il prodotto tra naturali del sorgente e %ax, scrive il risultato su %dx_%ax. Se il risultato non è riducibile a 16 bit, mette CF e OF a 1, altrimenti a 0.    |
| mul        | Unsigned Multiply, 32 bit | mull r/m  | Calcola su 64 bit il prodotto tra naturali del sorgente e %eax, scrive il risultato su %edx_%eax. Se il risultato non è riducibile a 32 bit, mette CF e OF a 1, altrimenti a 0. |
| imul       | Signed Multiply, 8 bit    | imulb r/m | Calcola su 16 bit il prodotto tra interi del sorgente e %al, scrive il risultato su %ax. Se il risultato non è riducibile a 8 bit, mette CF e OF a 1, altrimenti a 0.           |
| imul       | Signed Multiply, 16 bit   | imulw r/m | Calcola su 32 bit il prodotto tra interi del sorgente e %ax, scrive il risultato su %dx_%ax. Se il risultato non è riducibile a 16 bit, mette CF e OF a 1, altrimenti a 0.      |
| imul       | Signed Multiply, 32 bit   | imull r/m | Calcola su 64 bit il prodotto tra interi del sorgente e %eax, scrive il risultato su %edx_%eax. Se il risultato non è riducibile a 32 bit, mette CF e OF a 1, altrimenti a 0.   |

| Istruzione | Nome esteso             | Notazione | Comportamento                                                                                                                                                                                                                                 |
|------------|-------------------------|-----------|-----------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------|
| div        | Unsigned Divide, 8 bit  | divb r/m  | Calcola su 8 bit la divisione tra naturali tra %ax (dividendo implicito) e il sorgente (divisore). Scrive il quoziente su %al e il resto su %ah. Se il quoziente non è rappresentabile su 8 bit, causa <i>crash del programma</i> .           |
| div        | Unsigned Divide, 16 bit | divw r/m  | Calcola su 16 bit la divisione tra naturali tra %dx_%ax (dividendo implicito) e il sorgente (divisore). Scrive il quoziente su %ax e il resto su %dx. Se il quoziente non è rappresentabile su 16 bit, causa <i>crash del programma</i> .     |
| div        | Unsigned Divide, 32 bit | divl r/m  | Calcola su 32 bit la divisione tra naturali tra %edx_%eax (dividendo implicito) e il sorgente (divisore). Scrive il quoziente su %eax e il resto su %edx. Se il quoziente non è rappresentabile su 32 bit, causa <i>crash del programma</i> . |
| idiv       | Signed Divide, 8 bit    | idivb r/m | Calcola su 8 bit la divisione tra interi tra %ax (dividendo implicito) e il sorgente (divisore). Scrive il quoziente su %al e il resto su %ah. Se il quoziente non è rappresentabile su 8 bit, causa <i>crash del programma</i> .             |
| idiv       | Signed Divide, 16 bit   | idivw r/m | Calcola su 16 bit la divisione tra interi tra %dx_%ax (dividendo implicito) e il sorgente (divisore). Scrive il quoziente su %ax e il resto su %dx. Se il quoziente non è rappresentabile su 16 bit, causa <i>crash del programma</i> .       |
| idiv       | Signed Divide, 32 bit   | idivl r/m | Calcola su 32 bit la divisione tra interi tra %edx_%eax (dividendo implicito) e il sorgente (divisore). Scrive il quoziente su %eax e il resto su %edx. Se il quoziente non è rappresentabile su 32 bit, causa <i>crash del programma</i> .   |

## 2.3 Logica binaria

Le seguenti istruzioni operano *bit a bit*: data per esempio la and, l'i-esimo bit del risultato è l'and logico tra gli i-esimi bit di sorgente e destinatario.

| Istruzione | Notazione      | Comportamento                                                                            |
|------------|----------------|------------------------------------------------------------------------------------------|
| not        | not[bwl] r/m   | Sostituisce il destinatario con la sua negazione.                                        |
| and        | and r/m/i, r/m | Calcola l'and logico tra sorgente e destinatario, scrive il risultato sul destinatario.  |
| or         | or r/m/i, r/m  | Calcola l'or logico tra sorgente e destinatario, scrive il risultato sul destinatario.   |
| xor        | xor r/m/i, r/m | Calcola lo xor logico tra sorgente e destinatario, scrive il risultato sul destinatario. |

## 2.4 Traslazione e Rotazione

| Istruzione | Nome esteso           | Notazione        | Comportamento                                                                                                                                                                                                                                                                                                                                                                                   |
|------------|-----------------------|------------------|-------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------|
| shl        | Shift Logical Left    | shl[bwl] i/r r/m | Sia $n$ l'operando sorgente, esegue lo shift a sinistra del destinatario $n$ volte, impostando a 0 gli $n$ bit meno significativi. In ciascuno shift, il bit più significativo viene lasciato in CF. Come registro sorgente si può utilizzare solo %cl. Il sorgente può essere omissso, in quel caso $n = 1$ .                                                                                  |
| sal        | Shift Arithmetic Left | sal[bwl] i/r r/m | Sia $n$ l'operando sorgente, esegue lo shift a sinistra del destinatario $n$ volte, impostando a 0 gli $n$ bit meno significativi. In ciascuno shift, il bit più significativo viene lasciato in CF. Se il bit più significativo ha cambiato valore almeno una volta, imposta OF a 1. Come registro sorgente si può utilizzare solo %cl. Il sorgente può essere omissso, in quel caso $n = 1$ . |

|     |                         |                  |                                                                                                                                                                                                                                                                                                                                                                                        |
|-----|-------------------------|------------------|----------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------|
| shr | Shift Logical Right     | shr[bwl] i/r r/m | Sia $n$ l'operando sorgente, esegue lo shift a destra del destinatario $n$ volte, impostando a 0 gli $n$ bit più significativi. In ciascuno shift, il bit meno significativo viene lasciato in CF. Come registro sorgente si può utilizzare solo %c <sub>L</sub> . Il sorgente può essere omissso, in quel caso $n = 1$ .                                                              |
| sar | Shift Arithmetic Right  | sar[bwl] i/r r/m | Sia $n$ l'operando sorgente e $s$ il valore del bit più significativo del destinatario, esegue lo shift a destra del destinatario $n$ volte, impostando a $s$ gli $n$ bit più significativi. In ciascuno shift, il bit meno significativo viene lasciato in CF. Come registro sorgente si può utilizzare solo %c <sub>L</sub> . Il sorgente può essere omissso, in quel caso $n = 1$ . |
| rol | Rotate Left             | rol[bwl] i/r r/m | Sia $n$ l'operando sorgente, esegue la rotazione a sinistra del destinatario $n$ volte. In ciascuna rotazione, il bit più significativo viene <i>sia</i> lasciato in CF <i>sia</i> ricopiato al posto del bit meno significativo. Come registro sorgente si può utilizzare solo %c <sub>L</sub> . Il sorgente può essere omissso, in quel caso $n = 1$ .                               |
| ror | Rotate Right            | ror[bwl] i/r r/m | Sia $n$ l'operando sorgente, esegue la rotazione a destra del destinatario $n$ volte. In ciascuna rotazione, il bit meno significativo viene <i>sia</i> lasciato in CF <i>sia</i> ricopiato al posto del bit più significativo. Come registro sorgente si può utilizzare solo %c <sub>L</sub> . Il sorgente può essere omissso, in quel caso $n = 1$ .                                 |
| rcl | Rotate with Carry Left  | rcl[bwl] i/r r/m | Sia $n$ l'operando sorgente, esegue la rotazione con carry a sinistra del destinatario $n$ volte. In ciascuna rotazione, il bit più significativo viene lasciato in CF, mentre il valore di CF viene ricopiato al posto del bit meno significativo. Come registro sorgente si può utilizzare solo %c <sub>L</sub> . Il sorgente può essere omissso, in quel caso $n = 1$ .             |
| rcr | Rotate with Carry Right | rcr[bwl] i/r r/m | Sia $n$ l'operando sorgente, esegue la rotazione con carry a destra del destinatario $n$ volte. In ciascuna rotazione, il bit meno significativo viene lasciato in CF, mentre il valore di CF viene ricopiato al posto del bit più significativo. Come registro sorgente si può utilizzare solo %c <sub>L</sub> . Il sorgente può essere omissso, in quel caso $n = 1$ .               |

## 2.5 Controllo di flusso

| Istruzione | Nome esteso           | Notazione | Comportamento                                                                                                                                                          |
|------------|-----------------------|-----------|------------------------------------------------------------------------------------------------------------------------------------------------------------------------|
| jmp        | Unconditional Jump    | jmp m/r   | Salta incondizionatamente all'indirizzo specificato.                                                                                                                   |
| call       | Call Procedure        | call m/r  | Chiamata a procedura all'indirizzo specificato. Salva l'indirizzo della prossima istruzione nello stack, così che il flusso corrente possa essere ripreso con una ret. |
| ret        | Return from Procedure | ret       | Ritorna a un flusso di esecuzione precedente, rimuovendo dallo stack l'indirizzo precedentemente salvato da una call.                                                  |

La tabella seguente elenca i salti condizionati. I salti condizionati usano i flag per determinare se la condizione di salto è vera. Per un uso sempre coerente, assicurarsi che l'istruzione di salto segua immediatamente una cmp, o altre istruzioni che non hanno modificano i flag dopo la cmp. Dati gli operandi della cmp e una condizione  $c$ , per esempio  $c = \text{"maggiore o uguale"}$ , la condizione è vera se destinatario  $c$  sorgente. Nella tabella che segue, quando ci si riferisce a un confronto fra sorgente e destinatario si intendono gli operandi della cmp precedente.

| Istruzione | Nome esteso              | Notazione           | Comportamento                                                      |
|------------|--------------------------|---------------------|--------------------------------------------------------------------|
| cmp        | Compare Two Operands     | cmp[bwl] r/m/i, r/m | Confronta i due operandi e aggiorna i flag di conseguenza.         |
| je         | Jump if Equal            | je m                | Salta se destinatario == sorgente.                                 |
| jne        | Jump if Not Equal        | jne m               | Salta se destinatario != sorgente.                                 |
| ja         | Jump if Above            | ja m                | Salta se, interpretandoli come naturali, destinatario > sorgente.  |
| jae        | Jump if Above or Equal   | jae m               | Salta se, interpretandoli come naturali, destinatario >= sorgente. |
| jb         | Jump if Below            | jb m                | Salta se, interpretandoli come naturali, destinatario < sorgente.  |
| jbe        | Jump if Below or Equal   | jbe m               | Salta se, interpretandoli come naturali, destinatario <= sorgente. |
| jg         | Jump if Greater          | jg m                | Salta se, interpretandoli come interi, destinatario > sorgente.    |
| jge        | Jump if Greater or Equal | jge m               | Salta se, interpretandoli come interi, destinatario >= sorgente.   |
| jl         | Jump if Less             | jl m                | Salta se, interpretandoli come interi, destinatario < sorgente.    |
| jle        | Jump if Less or Equal    | jle m               | Salta se, interpretandoli come interi, destinatario <= sorgente.   |
| jz         | Jump if Zero             | jz m                | Salta se ZF è 1.                                                   |
| jnz        | Jump if Not Zero         | jnz m               | Salta se ZF è 0.                                                   |
| jc         | Jump if Carry            | jc m                | Salta se CF è 1.                                                   |
| jnc        | Jump if Not Carry        | jnc m               | Salta se CF è 0.                                                   |
| jo         | Jump if Overflow         | jo m                | Salta se OF è 1.                                                   |

|     |                      |       |                  |
|-----|----------------------|-------|------------------|
| jno | Jump if Not Overflow | jno m | Salta se OF è 0. |
| js  | Jump if Sign         | js m  | Salta se SF è 1. |
| jns | Jump if Not Sign     | jns m | Salta se SF è 0. |

## 2.6 Operazioni condizionali

Per alcune operazioni tipiche, sono disponibili istruzioni specifiche il cui comportamento dipende dai flag e, quindi, dal risultato di una precedente `cmp`. Anche qui, quando ci si riferisce a un confronto fra sorgente e destinatario si intendono gli operandi della `cmp` precedente.

La famiglia di istruzioni `loop` supporta i cicli condizionati più tipici. Rimangono d'interesse didattico come istruzioni specializzate ma, curiosamente, nei processori moderni sono generalmente meno performanti degli equivalenti che usino `dec`, `cmp` e salti condizionati.

| Istruzione | Nome esteso        | Notazione | Comportamento                                                                                                                                        |
|------------|--------------------|-----------|------------------------------------------------------------------------------------------------------------------------------------------------------|
| loop       | Unconditional Loop | loop m    | Decrementa <code>%ecx</code> e salta se il risultato è (ancora) diverso da 0.                                                                        |
| loope      | Loop if Equal      | loope m   | Decrementa <code>%ecx</code> e salta se entrambe le condizioni sono vere: 1) <code>%ecx</code> è (ancora) diverso da 0, 2) destinatario == sorgente. |
| loopne     | Loop if Not Equal  | loopne m  | Decrementa <code>%ecx</code> e salta se entrambe le condizioni sono vere: 1) <code>%ecx</code> è (ancora) diverso da 0, 2) destinatario != sorgente. |
| loopz      | Loop if Zero       | loopz m   | Decrementa <code>%ecx</code> e salta se entrambe le condizioni sono vere: 1) <code>%ecx</code> è (ancora) diverso da 0, 2) ZF è 1.                   |
| loopnz     | Loop if Not Zero   | loopnz m  | Decrementa <code>%ecx</code> e salta se entrambe le condizioni sono vere: 1) <code>%ecx</code> è (ancora) diverso da 0, 2) ZF è 0.                   |

La famiglia di istruzioni `set` permette di salvare il valore di un confronto in un registro o locazione di memoria. Tale operando può essere solo da 1 byte.

| Istruzione | Nome esteso             | Notazione | Comportamento                                                                                       |
|------------|-------------------------|-----------|-----------------------------------------------------------------------------------------------------|
| sete       | Set if Equal            | sete r/m  | Imposta l'operando a 1 se destinatario == sorgente, a 0 altrimenti.                                 |
| setne      | Set if Not Equal        | setne r/m | Imposta l'operando a 1 se destinatario != sorgente, a 0 altrimenti.                                 |
| seta       | Set if Above            | seta r/m  | Imposta l'operando a 1 se, interpretandoli come naturali, destinatario > sorgente, a 0 altrimenti.  |
| setae      | Set if Above or Equal   | setae r/m | Imposta l'operando a 1 se, interpretandoli come naturali, destinatario >= sorgente, a 0 altrimenti. |
| setb       | Set if Below            | setb r/m  | Imposta l'operando a 1 se, interpretandoli come naturali, destinatario < sorgente, a 0 altrimenti.  |
| setbe      | Set if Below or Equal   | setbe r/m | Imposta l'operando a 1 se, interpretandoli come naturali, destinatario <= sorgente, a 0 altrimenti. |
| setg       | Set if Greater          | setg r/m  | Imposta l'operando a 1 se, interpretandoli come interi, destinatario > sorgente, a 0 altrimenti.    |
| setge      | Set if Greater or Equal | setge r/m | Imposta l'operando a 1 se, interpretandoli come interi, destinatario >= sorgente, a 0 altrimenti.   |
| setl       | Set if Less             | setl r/m  | Imposta l'operando a 1 se, interpretandoli come interi, destinatario < sorgente, a 0 altrimenti.    |
| setle      | Set if Less or Equal    | setle r/m | Imposta l'operando a 1 se, interpretandoli come interi, destinatario <= sorgente, a 0 altrimenti.   |
| setz       | Set if Zero             | setz r/m  | Imposta l'operando a 1 se ZF è 1, a 0 altrimenti.                                                   |
| setnz      | Set if Not Zero         | setnz r/m | Imposta l'operando a 1 se ZF è 0, a 0 altrimenti.                                                   |
| setc       | Set if Carry            | setc r/m  | Imposta l'operando a 1 se CF è 1, a 0 altrimenti.                                                   |
| setnc      | Set if Not Carry        | setnc r/m | Imposta l'operando a 1 se CF è 0, a 0 altrimenti.                                                   |
| seto       | Set if Overflow         | seto r/m  | Imposta l'operando a 1 se OF è 1, a 0 altrimenti.                                                   |
| setno      | Set if Not Overflow     | setno r/m | Imposta l'operando a 1 se OF è 0, a 0 altrimenti.                                                   |
| sets       | Set if Sign             | sets r/m  | Imposta l'operando a 1 se SF è 1, a 0 altrimenti.                                                   |
| setns      | Set if Not Sign         | setns r/m | Imposta l'operando a 1 se SF è 0, a 0 altrimenti.                                                   |

La famiglia di istruzioni `cmov` permette di eseguire, solo se il confronto ha avuto successo, una `mov` da memoria a registro o da registro a registro. Gli operandi possono essere solo a 2 o 4 byte, non 1.

| Istruzione | Nome esteso       | Notazione        | Comportamento                                                                    |
|------------|-------------------|------------------|----------------------------------------------------------------------------------|
| cmov       | Move if Equal     | cmov[wl] r/m r   | Esegue la <code>mov</code> se destinatario == sorgente, altrimenti non fa nulla. |
| cmovne     | Move if Not Equal | cmovne[wl] r/m r | Esegue la <code>mov</code> se destinatario != sorgente, altrimenti non fa nulla. |



|        |                          |                  |                                                                                                     |
|--------|--------------------------|------------------|-----------------------------------------------------------------------------------------------------|
| cmova  | Move if Above            | cmova[wl] r/m r  | Esegue la mov se, interpretandoli come naturali, destinatario > sorgente, altrimenti non fa nulla.  |
| cmovae | Move if Above or Equal   | cmovae[wl] r/m r | Esegue la mov se, interpretandoli come naturali, destinatario >= sorgente, altrimenti non fa nulla. |
| cmovb  | Move if Below            | cmovb[wl] r/m r  | Esegue la mov se, interpretandoli come naturali, destinatario < sorgente, altrimenti non fa nulla.  |
| cmovbe | Move if Below or Equal   | cmovbe[wl] r/m r | Esegue la mov se, interpretandoli come naturali, destinatario <= sorgente, altrimenti non fa nulla. |
| cmovg  | Move if Greater          | cmovg[wl] r/m r  | Esegue la mov se, interpretandoli come interi, destinatario > sorgente, altrimenti non fa nulla.    |
| cmovge | Move if Greater or Equal | cmovge[wl] r/m r | Esegue la mov se, interpretandoli come interi, destinatario >= sorgente, altrimenti non fa nulla.   |
| cmovl  | Move if Less             | cmovl[wl] r/m r  | Esegue la mov se, interpretandoli come interi, destinatario < sorgente, altrimenti non fa nulla.    |
| cmovle | Move if Less or Equal    | cmovle[wl] r/m r | Esegue la mov se, interpretandoli come interi, destinatario <= sorgente, altrimenti non fa nulla.   |
| cmovz  | Move if Zero             | cmovz[wl] r/m r  | Esegue la mov se ZF è 1, altrimenti non fa nulla.                                                   |
| cmovnz | Move if Not Zero         | cmovnz[wl] r/m r | Esegue la mov se ZF è 0, altrimenti non fa nulla.                                                   |
| cmovc  | Move if Carry            | cmovc[wl] r/m r  | Esegue la mov se CF è 1, altrimenti non fa nulla.                                                   |
| cmovnc | Move if Not Carry        | cmovnc[wl] r/m r | Esegue la mov se CF è 0, altrimenti non fa nulla.                                                   |
| cmovo  | Move if Overflow         | cmovo[wl] r/m r  | Esegue la mov se OF è 1, altrimenti non fa nulla.                                                   |
| cmovno | Move if Not Overflow     | cmovno[wl] r/m r | Esegue la mov se OF è 0, altrimenti non fa nulla.                                                   |
| cmovs  | Move if Sign             | cmovs[wl] r/m r  | Esegue la mov se SF è 1, altrimenti non fa nulla.                                                   |
| cmovns | Move if Not Sign         | cmovns[wl] r/m r | Esegue la mov se SF è 0, altrimenti non fa nulla.                                                   |

## 2.7 Istruzioni stringa

Le istruzioni stringa sono ottimizzate per eseguire operazioni tipiche su vettori in memoria. Hanno esclusivamente operandi impliciti, che rende la specifica delle dimensioni *non* opzionale.

| Istruzione | Nome esteso           | Notazione | Comportamento                                                                                                                                       |
|------------|-----------------------|-----------|-----------------------------------------------------------------------------------------------------------------------------------------------------|
| cld        | Clear Direction Flag  | cld       | Imposta DF a 0, implicando che le istruzioni stringa procederanno per indirizzi crescenti.                                                          |
| std        | Set Direction Flag    | std       | Imposta DF a 1, implicando che le istruzioni stringa procederanno per indirizzi decrescenti.                                                        |
| lods       | Load String           | lods[bwl] | Legge 1/2/4 byte all'indirizzo in %esi e lo scrive in %al / %ax / %eax. Se DF è 0, incrementa %esi di 1/2/4, se è 1 lo decrementa.                  |
| stos       | Store String          | stos[bwl] | Legge il valore in %al / %ax / %eax e lo scrive nei 1/2/4 byte all'indirizzo in %edi. Se DF è 0, incrementa %edi di 1/2/4, se è 1 lo decrementa.    |
| movs       | Move String to String | movs[bwl] | Legge 1/2/4 byte all'indirizzo in %esi e lo scrive nei 1/2/4 byte all'indirizzo in %edi. Se DF è 0, incrementa %edi di 1/2/4, se è 1 lo decrementa. |
| cmps       | Compare Strings       | cmps[bwl] | Confronta gli 1/2/4 byte all'indirizzo in %esi (sorgente) con quelli all'indirizzo in %edi (destinatario). Aggiorna i flag così come fa cmp.        |
| scas       | Scan String           | scas[bwl] | Confronta %al / %ax / %eax (sorgente) con gli 1/2/4 byte all'indirizzo in %edi (destinatario). Aggiorna i flag così come fa cmp.                    |

### Repeat Instruction

Le istruzioni stringa possono essere ripetute senza controllo di programma, usando il prefisso rep.

| Istruzione | Nome esteso                      | Notazione      | Comportamento                                                                                                                                                                                        |
|------------|----------------------------------|----------------|------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------|
| rep        | Unconditional Repeat Instruction | rep [opcode]   | Dato n il valore in %ecx, ripete l'operazione opcode n volte, decrementando %ecx fino a 0. Compatibile con lods, stos, movs.                                                                         |
| repe       | Repeat Instruction if Equal      | repe [opcode]  | Dato n il valore in %ecx, decrementa %ecx e ripete l'operazione opcode finché 1) %ecx è (ancora) diverso da 0, e 2) gli operandi di questa ripetizione erano uguali. Compatibile con cmps e scas.    |
| repne      | Repeat Instruction if Not Equal  | repne [opcode] | Dato n il valore in %ecx, decrementa %ecx e ripete l'operazione opcode finché 1) %ecx è (ancora) diverso da 0, e 2) gli operandi di questa ripetizione erano disuguali. Compatibile con cmps e scas. |

## 2.8 Altre istruzioni

| Istruzione | Nome esteso  | Notazione | Comportamento                                                                   |
|------------|--------------|-----------|---------------------------------------------------------------------------------|
| nop        | No Operation | nop       | Non cambia lo stato del processore in alcun modo, eccetto per il registro %eip. |

Le seguenti istruzioni sono di interesse didattico ma non per le esercitazioni, in quanto richiedono privilegi di esecuzione.

| Istruzione | Nome esteso            | Notazione | Comportamento                                                                                           |
|------------|------------------------|-----------|---------------------------------------------------------------------------------------------------------|
| in         | Input from Port        | in r/i r  | Legge da una porta di input a un registro.                                                              |
| out        | Output to Port         | out r r/i | Scrive da un registro a una porta di output.                                                            |
| ins        | Input String from Port | ins[bwl]  | Legge 1/2/4 byte dalla porta di input indicata in %dx e li scrive nei 1/2/4 byte all'indirizzo in %edi. |
| outs       | Output String to Port  | outs[bwl] | Legge 1/2/4 byte all'indirizzo indicato da %esi e li scrive alla porta di output indicata in %dx.       |
| hlt        | Halt                   | hlt       | Blocca ogni operazione del processore.                                                                  |

## 3. Sottoprogrammi di utility

Nell'architettura del processore, menzioniamo registri, istruzioni e locazioni di memoria. Quando scriviamo programmi, sfruttiamo però il concetto di *terminale*, un'interfaccia dove l'utente legge caratteri e ne scrive usando la tastiera. Come questo possa avvenire è argomento di altri corsi, dove verranno presentate le *interruzioni*, il *kernel*, e in generale cosa fa un *sistema operativo*.

In questo corso ci limitiamo a sfruttare queste funzionalità tramite del codice ad hoc contenuto in `utility.s`. Queste funzionalità sono fornite come sottoprogrammi, che hanno i loro specifici comportamenti da tenere a mente.

Per utilizzare questi sottoprogrammi, utilizziamo la direttiva

```
.include "../files/utility.s"
```

### 3.1 Terminologia

Con *leggere caratteri da tastiera* si intende che il programma resta in attesa che l'utente prema un tasto sulla tastiera, inviando la codifica di quel tasto al programma.

Con *mostrare a terminale* si intende che il programma stampa un carattere a video.

Con *fare eco* di un carattere si intende che il programma, subito dopo aver letto un carattere da tastiera, lo mostra anche a schermo. Questo è il comportamento interattivo a cui siamo più abituati, ma non è automatico.

Con *ignorare caratteri* si intende che il programma, dopo aver letto un carattere, controlli che questo sia del tipo atteso: se lo è ne fa eco o comunque risponde in modo interattivo, se non lo è ritorna in lettura di un altro carattere, mostrandosi all'utente come se avesse, appunto, ignorato il carattere precedente.

### 3.2 Caratteri speciali

Avanzamento linea ( *line feed*, LF): carattere `\n`, codifica `0x0A`.

Ritorno carrello ( *carriage return*, CR): carattere `\r`, codifica `0x0D`.

Il significato di questi ha a che vedere con le macchine da scrivere, dove *avanzare alla riga successiva* e *riportare il carrello a sinistra* erano azioni ben distinte.

### 3.3 Sottoprogrammi

| Nome                                                                                       | Comportamento                                                                                                                                                                                                                                                                                                     |
|--------------------------------------------------------------------------------------------|-------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------|
| <code>inchar</code>                                                                        | Legge da tastiera un carattere ASCII e ne scrive la codifica in <code>%a\</code> . Non mostra a terminale il carattere letto.                                                                                                                                                                                     |
| <code>outchar</code>                                                                       | Legge la codifica di un carattere ASCII dal registro <code>%a\</code> e lo mostra a terminale.                                                                                                                                                                                                                    |
| <code>inbyte</code> / <code>inword</code> / <code>inlong</code>                            | Legge dalla tastiera 2/4/8 cifre esadecimali (0-9 e A-F), facendone eco e ignorando altri caratteri. Salva quindi il byte/word/long corrispondente a tali cifre in <code>%a\</code> / <code>%ax</code> / <code>%eax</code> .                                                                                      |
| <code>outbyte</code> / <code>outword</code> / <code>outlong</code>                         | Legge il contenuto di <code>%a\</code> / <code>%ax</code> / <code>%eax</code> e lo mostra a terminale sotto forma di 2/4/8 cifre esadecimali.                                                                                                                                                                     |
| <code>indecimal_byte</code> / <code>indecimal_word</code> / <code>indecimal_long</code>    | Legge dalla tastiera fino a 3/5/10 cifre decimali (0-9), o finché non è inserito un <code>\r</code> , facendone eco e ignorando altri caratteri. Interpreta queste come cifre di un numero naturale, e salva quindi il byte/word/long corrispondente in <code>%a\</code> / <code>%ax</code> / <code>%eax</code> . |
| <code>outdecimal_byte</code> / <code>outdecimal_word</code> / <code>outdecimal_long</code> | Legge il contenuto di <code>%a\</code> / <code>%ax</code> / <code>%eax</code> , lo interpreta come numero naturale e lo mostra a terminale sotto forma di cifre decimali.                                                                                                                                         |
| <code>outmess</code>                                                                       | Dato l'indirizzo <code>v</code> in <code>%ebx</code> e il numero <code>n</code> in <code>%cx</code> , mostra a terminale gli <code>n</code> caratteri ASCII memorizzati a partire da <code>v</code> .                                                                                                             |

|         |                                                                                                                                                                                                                                                                                      |
|---------|--------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------|
| outline | Dato l'indirizzo $v$ in %ebx, mostra a terminale i caratteri ASCII memorizzati a partire da $v$ finché non incontra un \r o raggiunge il massimo di 80 caratteri.                                                                                                                    |
| inline  | Dato l'indirizzo $v$ in %ebx e il numero $n$ in %cx, legge da tastiera caratteri ASCII e li scrive a partire da $v$ finché non è inserito un \r o raggiunge il massimo di $n - 2$ caratteri. Pone poi in fondo i caratteri \r\n. Supporta l'uso di backspace per correggere l'input. |
| newline | Porta l'output del terminale a una nuova riga, mostrando i caratteri \r\n.                                                                                                                                                                                                           |

## 4. Debugger gdb

gdb è un debugger a linea di comando che ci permette di eseguire un programma passo passo, seguendo lo stato del processore e della memoria.

Il concetto fondamentale per un debugger è quello di *breakpoint*, ossia un punto del codice dove l'esecuzione dovrà fermarsi. I breakpoints ci permettono di eseguire rapidamente le parti del programma che non sono di interesse e fermarsi ad osservare solo le parti che ci interessano.

Quella che segue è comunque una presentazione sintetica e semplificata. Per altre opzioni e funzionalità del debugger, vedere la documentazione ufficiale o il comando `help`.

### 4.1 Controllo dell'esecuzione

Per istruzione corrente si intende *la prossima da eseguire*. Quando il debugger si ferma a un'istruzione, si ferma *prima* di eseguirla.

| Nome completo | Nome scorciatoia | Formato        | Comportamento                                                                                                                                                                                                                                                                                                                                        |
|---------------|------------------|----------------|------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------|
| frame         | f                | f              | Mostra l'istruzione corrente.                                                                                                                                                                                                                                                                                                                        |
| list          | l                | l              | Mostra il sorgente attorno all'istruzione corrente.                                                                                                                                                                                                                                                                                                  |
| break         | b                | b <i>label</i> | Imposta un breakpoint alla prima istruzione dopo <i>label</i> .                                                                                                                                                                                                                                                                                      |
| continue      | c                | c              | Prosegue l'esecuzione del programma fino al prossimo breakpoint.                                                                                                                                                                                                                                                                                     |
| step          | s                | s              | Esegue l'istruzione corrente, fermandosi immediatamente dopo. Se l'istruzione corrente è una <code>call</code> , l'esecuzione si fermerà alla prima istruzione del sottoprogramma chiamato.                                                                                                                                                          |
| next          | n                | n              | Esegue l'istruzione corrente, fermandosi all'istruzione successiva del sottoprogramma corrente. Se l'istruzione corrente è una <code>call</code> , l'esecuzione si fermerà <i>dopo</i> il <code>ret</code> di del sottoprogramma chiamato. Nota: aggiungere una <code>nop</code> dopo ogni <code>call</code> prima di una nuova <code>label</code> . |
| finish        | fin              | fin            | Continua l'esecuzione fino all'uscita dal sottoprogramma corrente ( <code>ret</code> ). L'esecuzione si fermerà alla prima istruzione dopo la <code>call</code> .                                                                                                                                                                                    |
| run           | r                | r              | Avvia (o riavvia) l'esecuzione del programma. Chiede conferma.                                                                                                                                                                                                                                                                                       |
| quit          | q                | q              | Esce dal debugger. Chiede conferma.                                                                                                                                                                                                                                                                                                                  |

I seguenti comandi sono *definiti ad hoc nell'ambiente del corso*, e non sono quindi tipici comandi di gdb.

| Nome completo | Nome scorciatoia | Formato | Comportamento                                                          |
|---------------|------------------|---------|------------------------------------------------------------------------|
| rrun          | rr               | rr      | Avvia (o riavvia) l'esecuzione del programma, senza chiedere conferma. |
| qquit         | qq               | qq      | Esce dal debugger, senza chiedere conferma.                            |

#### Problemi con next

Si possono talvolta incontrare problemi con il comportamento di `next`, che derivano da come questa è definita e implementata. Il comando `next` distingue i *frame* come le sequenze di istruzioni che vanno da una *label* alla successiva. Il suo comportamento è, in realtà, di continuare l'esecuzione finché non incontra di nuovo una nuova istruzione nello stesso *frame* di partenza.

Questa logica può essere facilmente rotta con del codice come il seguente, dove *non esiste* una istruzione di `punto_1` che viene incontrata dopo la `call`. Quel che ne consegue è che il comando `next` si comporta come `continue`.

```
punto_1:
...
    call newline
punto_2:
...
```

Per ovviare a questo problema, è una buona abitudine quella di aggiungere una `nop` dopo ciascuna `call`. Tale `nop`, appartenendo allo stesso *frame* `punto_1`, farà regolarmente sospendere l'esecuzione.

```
punto_1:
...
    call newline
    nop
punto_2:
...
```

## 4.2 Ispezione dei registri

| Nome completo  | Nome scorciatoia | Formato | Comportamento                                                                                                                                                                                                                        |
|----------------|------------------|---------|--------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------|
| info registers | i r              | i r     | Mostra lo stato di (quasi) tutti i registri. Non mostra separatamente i sotto-registri, come %ax.                                                                                                                                    |
| info registers | i r              | i r reg | Mostra lo stato del registro <i>reg</i> specificato. <i>reg</i> va specificato in minuscolo senza caratteri preposti, per esempio i r eax. Si possono specificare anche sotto-registri, come %ax, e più registri separati da spazio. |

gdb supporta viste alternative con il comando `layout` che mettono più informazioni a schermo. In particolare, `layout regs` mostra l'equivalente di `i r` e `l`, evidenziando gli elementi che cambiano ad ogni step di esecuzione.

## 4.3 Ispezione della memoria

| Nome completo | Nome scorciatoia | Formato     | Comportamento                                                                                                                                                                                                                                                                             |
|---------------|------------------|-------------|-------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------|
| x             | x                | x/ NFU addr | Mostra lo stato della memoria a partire dall'indirizzo <i>addr</i> , per le <i>N</i> locazioni di dimensione <i>U</i> e interpretate con il formato <i>F</i> . Comando con memoria, i valori di <i>N</i> , <i>F</i> e <i>U</i> possono essere omessi (insieme allo / ) se uguali a prima. |

Il comando `x` sta per *examine memory*, ma a differenza degli altri non ha una versione estesa.

Il parametro *N* si specifica come un numero intero, il valore di default (all'avvio di gdb) è 1.

Il parametro *F* può essere

- x per esadecimale
- d per decimale
- c per ASCII
- t per binario
- s per stringa delimitata da 0x00

Il valore di default (all'avvio di gdb) è x.

Il parametro *U* può essere

- b per byte
- h per word (2 byte)
- w per long (4 byte)

Il valore di default (all'avvio di gdb) è h.

L'argomento *addr* può essere espresso in diversi modi, sia usando label che registri o espressioni basate su aritmetica dei puntatori. Per esempio:

- letterale esadecimale: `x 0x56559066`
- label: `x &label`
- registro puntatore: `x $esi`
- registro puntatore e registro indice: `x (char*)$esi + $ecx`

Notare che nell'ultimo caso, dato che ci si basa su aritmetica dei puntatori, il tipo all'interno del cast determina la *scala*, ossia la dimensione di ciascuna delle *\$ecx* locazioni del vettore da saltare. Si può usare `(char*)` per 1 byte, `(short*)` per 2 byte, `(int*)` per 4 byte.

Un'alternativa a questo è lo scomporre, anche solo temporaneamente, le istruzioni con indirizzamento complesso. Per esempio, si può sostituire `movb (%esi, %ecx), %al` con `lea (%esi, %ecx), %ebx` seguita da `movb (%ebx), %al`, così che si possa eseguire semplicemente `x $ebx` nel debugger.

## 4.4 Gestione dei breakpoints

Oltre a crearli, i breakpoint possono anche essere rimossi o (dis)abilitati. Questi comandi si basano sulla conoscenza dell' *id* di un breakpoint: questo viene stampato quando un breakpoint viene creato o raggiunto durante l'esecuzione, oppure si possono ristampare tutti usando `info b`.

| Nome completo                    | Nome scorciatoia    | Formato                    | Comportamento                                                                    |
|----------------------------------|---------------------|----------------------------|----------------------------------------------------------------------------------|
| <code>info breakpoints</code>    | <code>info b</code> | <code>info b [ id ]</code> | Stampa informazioni sul breakpoint <i>id</i> , o tutti se l'argomento è omissso. |
| <code>disable breakpoints</code> | <code>dis</code>    | <code>dis [ id ]</code>    | Disabilita il breakpoint <i>id</i> , o tutti se l'argomento è omissso.           |
| <code>enable breakpoints</code>  | <code>en</code>     | <code>en [ id ]</code>     | Abilita il breakpoint <i>id</i> , o tutti se l'argomento è omissso.              |
| <code>delete breakpoints</code>  | <code>d</code>      | <code>d [ id ]</code>      | Rimuove il breakpoint <i>id</i> , o tutti se l'argomento è omissso.              |

### Conditional Breakpoints

In alcuni casi, la complessità del programma, l'uso intensivo di sottoprogrammi o lunghi loop possono rendere molto lungo trovare il punto giusto dell'esecuzione. A questo scopo, è possibile definire dei *breakpoint condizionali*, per far sì che l'esecuzione si interrompa a tale breakpoint solo se la condiziona è verificata.

| Nome completo          | Nome scorciatoia  | Formato                   | Comportamento                                                   |
|------------------------|-------------------|---------------------------|-----------------------------------------------------------------|
| <code>condition</code> | <code>cond</code> | <code>cond id cond</code> | Imposta la condizione <i>cond</i> per il breakpoint <i>id</i> . |

La sintassi per una condizione è in "stile C", come il comando `x`. Alcuni esempi di questa sintassi:

- `cond 2 $a1==5` per far sì che l'esecuzione si fermi al breakpoint 2 solo se il registro `a1` contiene il valore 5;
- `cond 2 (short *)$edi==-5` per far sì che l'esecuzione si fermi al breakpoint 2 solo se il registro `edi` contiene l'indirizzo di una word di valore -5;
- `cond 2 (int *)&count!=0` per far sì che l'esecuzione si fermi al breakpoint 2 solo se la locazione di 4 byte a partire da `count` contiene un valore diverso da 0.

Fare attenzione alle conversioni automatiche di rappresentazione: quando si usa la rappresentazione decimale, `gdb` interpreta automaticamente i valori come interi. Una condizione come `cond 2 $a1==128`, per quanto accettata dal debugger, sarà sempre falsa perché la codifica `0x80` è interpretata in decimale come l'intero -128, mai come il naturale 128. È quindi una buona idea usare la notazione esadecimale in casi del genere, cioè quando il bit più significativo è 1.

Una feature disponibile in molti IDE è quello di creare dipendenze tra breakpoint, cioè abilitare un breakpoint solo se è stato prima colpito un altro. Questo però è [fin troppo ostico](#) da fare in `gdb`.

### Watchpoints

I watchpoint sono come dei breakpoint ma per dati (registri e memoria), non per il codice. Si creano indicando l'espressione del dato da controllare. Si gestiscono *con gli stessi comandi per i breakpoint*.

| Nome completo                    | Nome scorciatoia      | Formato                      | Comportamento                                                                       |
|----------------------------------|-----------------------|------------------------------|-------------------------------------------------------------------------------------|
| <code>watchpoint</code>          | <code>watch</code>    | <code>watch expr</code>      | Imposta un watchpoint per l'espressione <i>expr</i> .                               |
| <code>info watchpoints</code>    | <code>info wat</code> | <code>info wat [ id ]</code> | Stampa informazioni sul watchpoint <i>id</i> , o tutti se l'argomento è omissso.    |
| <code>disable breakpoints</code> | <code>dis</code>      | <code>dis [ id ]</code>      | Disabilita il breakpoint o watchpoint <i>id</i> , o tutti se l'argomento è omissso. |
| <code>enable breakpoints</code>  | <code>en</code>       | <code>en [ id ]</code>       | Abilita il breakpoint o watchpoint <i>id</i> , o tutti se l'argomento è omissso.    |
| <code>delete breakpoints</code>  | <code>d</code>        | <code>d [ id ]</code>        | Rimuove il breakpoint o watchpoint <i>id</i> , o tutti se l'argomento è omissso.    |

Un watchpoint richiede la specifica di un registro o locazione nella stessa notazione "stile C" del comando `x`, e interrompe l'esecuzione quando tale valore cambia. Per esempio, `watch $eax` crea un watchpoint che interrompe l'esecuzione ogni volta che `eax` cambia valore.





## 5. Tabella ASCII

Dalla tabella seguente sono esclusi caratteri non stampabili che non sono di nostro interesse.

| Codifica binaria | Codifica decimale | Codifica esadecimale | Carattere                 |
|------------------|-------------------|----------------------|---------------------------|
| 0000 0000        | 00                | 0x00                 | \0                        |
| 0000 1000        | 08                | 0x08                 | backspace                 |
| 0000 1001        | 09                | 0x09                 | \t, Horizontal Tabulation |
| 0000 1010        | 10                | 0x0A                 | \n, Line Feed             |
| 0000 1101        | 13                | 0x0D                 | \r, Carriage Return       |
| 0010 0000        | 32                | 0x20                 | space                     |
| 0010 0001        | 33                | 0x21                 | !                         |
| 0010 0010        | 34                | 0x22                 | "                         |
| 0010 0011        | 35                | 0x23                 | #                         |
| 0010 0100        | 36                | 0x24                 | \$                        |
| 0010 0101        | 37                | 0x25                 | %                         |
| 0010 0110        | 38                | 0x26                 | &                         |
| 0010 0111        | 39                | 0x27                 | '                         |
| 0010 1000        | 40                | 0x28                 | (                         |
| 0010 1001        | 41                | 0x29                 | )                         |
| 0010 1010        | 42                | 0x2A                 | *                         |
| 0010 1011        | 43                | 0x2B                 | +                         |
| 0010 1100        | 44                | 0x2C                 | ,                         |
| 0010 1101        | 45                | 0x2D                 | -                         |
| 0010 1110        | 46                | 0x2E                 | .                         |
| 0010 1111        | 47                | 0x2F                 | /                         |
| 0011 0000        | 48                | 0x30                 | 0                         |
| 0011 0001        | 49                | 0x31                 | 1                         |
| 0011 0010        | 50                | 0x32                 | 2                         |
| 0011 0011        | 51                | 0x33                 | 3                         |
| 0011 0100        | 52                | 0x34                 | 4                         |
| 0011 0101        | 53                | 0x35                 | 5                         |
| 0011 0110        | 54                | 0x36                 | 6                         |
| 0011 0111        | 55                | 0x37                 | 7                         |
| 0011 1000        | 56                | 0x38                 | 8                         |
| 0011 1001        | 57                | 0x39                 | 9                         |
| 0011 1010        | 58                | 0x3A                 | :                         |
| 0011 1011        | 59                | 0x3B                 | ;                         |
| 0011 1100        | 60                | 0x3C                 | <                         |
| 0011 1101        | 61                | 0x3D                 | =                         |
| 0011 1110        | 62                | 0x3E                 | >                         |
| 0011 1111        | 63                | 0x3F                 | ?                         |
| 0100 0000        | 64                | 0x40                 | @                         |
| 0100 0001        | 65                | 0x41                 | A                         |
| 0100 0010        | 66                | 0x42                 | B                         |
| 0100 0011        | 67                | 0x43                 | C                         |
| 0100 0100        | 68                | 0x44                 | D                         |
| 0100 0101        | 69                | 0x45                 | E                         |
| 0100 0110        | 70                | 0x46                 | F                         |
| 0100 0111        | 71                | 0x47                 | G                         |
| 0100 1000        | 72                | 0x48                 | H                         |
| 0100 1001        | 73                | 0x49                 | I                         |
| 0100 1010        | 74                | 0x4A                 | J                         |
| 0100 1011        | 75                | 0x4B                 | K                         |
| 0100 1100        | 76                | 0x4C                 | L                         |
| 0100 1101        | 77                | 0x4D                 | M                         |
| 0100 1110        | 78                | 0x4E                 | N                         |
| 0100 1111        | 79                | 0x4F                 | O                         |
| 0101 0000        | 80                | 0x50                 | P                         |

|           |     |      |   |
|-----------|-----|------|---|
| 0101 0001 | 81  | 0x51 | Q |
| 0101 0010 | 82  | 0x52 | R |
| 0101 0011 | 83  | 0x53 | S |
| 0101 0100 | 84  | 0x54 | T |
| 0101 0101 | 85  | 0x55 | U |
| 0101 0110 | 86  | 0x56 | V |
| 0101 0111 | 87  | 0x57 | W |
| 0101 1000 | 88  | 0x58 | X |
| 0101 1001 | 89  | 0x59 | Y |
| 0101 1010 | 90  | 0x5A | Z |
| 0101 1011 | 91  | 0x5B | [ |
| 0101 1100 | 92  | 0x5C | \ |
| 0101 1101 | 93  | 0x5D | ] |
| 0101 1110 | 94  | 0x5E | ^ |
| 0101 1111 | 95  | 0x5F | _ |
| 0110 0000 | 96  | 0x60 | ` |
| 0110 0001 | 97  | 0x61 | a |
| 0110 0010 | 98  | 0x62 | b |
| 0110 0011 | 99  | 0x63 | c |
| 0110 0100 | 100 | 0x64 | d |
| 0110 0101 | 101 | 0x65 | e |
| 0110 0110 | 102 | 0x66 | f |
| 0110 0111 | 103 | 0x67 | g |
| 0110 1000 | 104 | 0x68 | h |
| 0110 1001 | 105 | 0x69 | i |
| 0110 1010 | 106 | 0x6A | j |
| 0110 1011 | 107 | 0x6B | k |
| 0110 1100 | 108 | 0x6C | l |
| 0110 1101 | 109 | 0x6D | m |
| 0110 1110 | 110 | 0x6E | n |
| 0110 1111 | 111 | 0x6F | o |
| 0111 0000 | 112 | 0x70 | p |
| 0111 0001 | 113 | 0x71 | q |
| 0111 0010 | 114 | 0x72 | r |
| 0111 0011 | 115 | 0x73 | s |
| 0111 0100 | 116 | 0x74 | t |
| 0111 0101 | 117 | 0x75 | u |
| 0111 0110 | 118 | 0x76 | v |
| 0111 0111 | 119 | 0x77 | w |
| 0111 1000 | 120 | 0x78 | x |
| 0111 1001 | 121 | 0x79 | y |
| 0111 1010 | 122 | 0x7A | z |
| 0111 1011 | 123 | 0x7B | { |
| 0111 1100 | 124 | 0x7C |   |
| 0111 1101 | 125 | 0x7D | } |
| 0111 1110 | 126 | 0x7E | ~ |

From <https://en.wikipedia.org/wiki/ASCII>

## 6. Ambiente d'esame e i suoi script

Qui di seguito sono documentati gli script dell'ambiente. I principali sono `assemble.ps1` e `debug.ps1`, il cui uso è mostrato nelle esercitazioni. Gli script `run-test.ps1` e `run-tests.ps1` sono utili per automatizzare i test, il loro uso è del tutto opzionale.

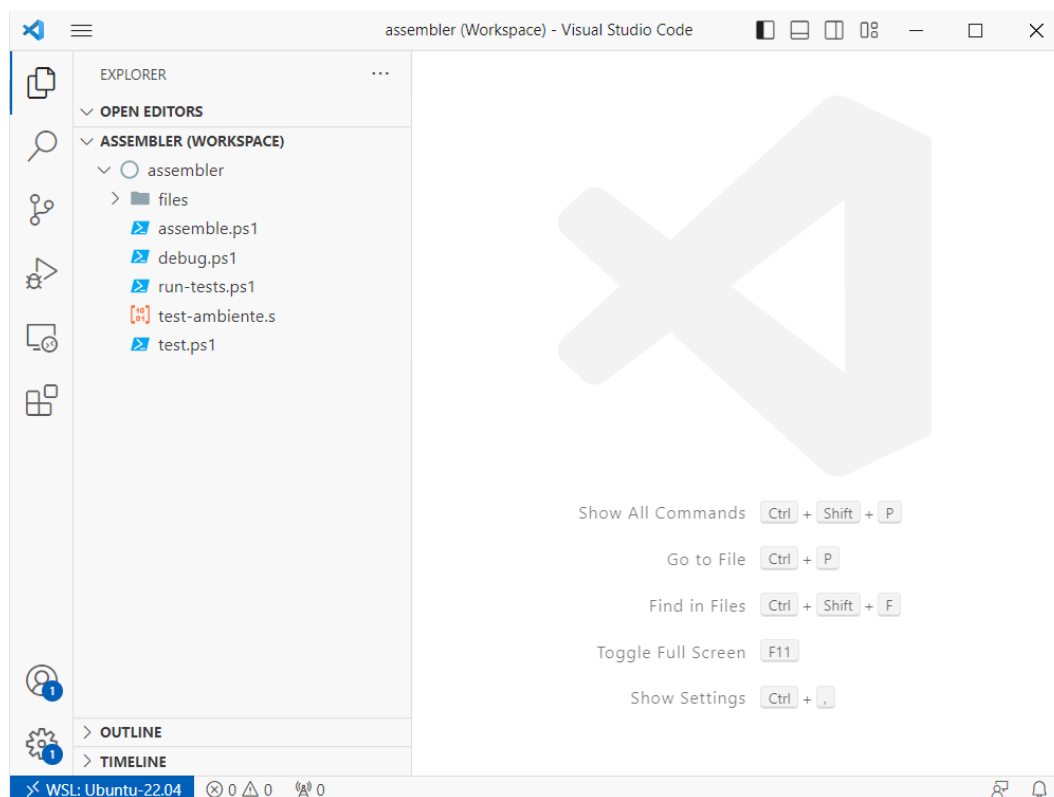
### 6.1 Aprire l'ambiente

Sulle macchine all'esame (o sulla propria, se si seguono tutti i passi indicati nel pacchetto di installazione) troverete una cartella `C:/reti_logiche` con contenuto come da figura.

- assembler
- dispense
- verilog
- assembler.code-workspace
- Istruzioni.txt
- verilog.code-workspace

Facendo doppio click sul file `assembler.code-workspace` verrà lanciato VS Code, collegandosi alla macchina virtuale WSL e la cartella di lavoro `C:/reti_logiche/assembler`.

La finestra VS Code che si aprirà sarà simile alla seguente.



Nell'angolo in basso a sinistra, `WSL: Ubuntu-22.04` sta a indicare che l'editor è correttamente connesso alla macchina virtuale.

I file e cartelle mostrati nell'immagine sono quelli che ci si deve aspettare dall'ambiente vuoto.

In caso si trovino file in più all'esame, si possono *cancellare*.

Il file `test-ambiente.s` è un semplice programma per verificare che l'ambiente funzioni. Il contenuto è il seguente:

```
.include "../files/utility.s"

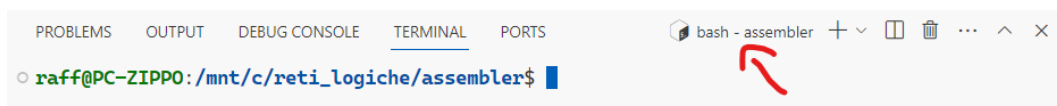
.data
messaggio: .ascii "Ok.\r"

.text
_main:
    nop
    lea messaggio, %ebx
    call outline
    ret
```

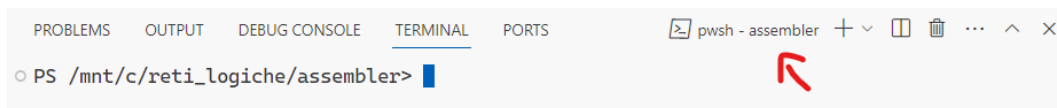
## 6.2 Il terminale Powershell

Per aprire un terminale in VS Code possiamo usare `Terminale -> Nuovo Terminale`. Per eseguire gli script dell'ambiente c'è bisogno di aprire un terminale *Powershell*. La shell standard di Linux, `bash`, non è in grado di eseguire questi script.

Non così:



Ma così:



Per cambiare shell si può usare il bottone `+` sulla destra, o lanciare il comando `pwsh` senza argomenti.

Se si preferisce, in VS Code si può aprire un terminale anche come tab dell'editor, o spostandolo al lato anziché in basso.

### Perché Powershell?

Perché Powershell (2006) è object-oriented, e permette di scrivere script leggibili e manutenibili, in modo semplice. Bash (1989) è invece text-oriented, con una [lunga lista di trappole da saper evitare](#).

## 6.3 Eseguire gli script

Gli script forniti permettono di assemblare, debuggare e testare il proprio programma. È importante che vengano eseguiti senza cambiare cartella, cioè non usando il comando `cd` o simili. Ricordarsi anche dei `./`, necessari per indicare al terminale che i file indicati vanno cercati nella cartella corrente.

Il tasto `tab` della tastiera invoca l'autocompletamento, che aiuta ad assicurarsi di inserire percorsi corretti.

Si ricorda inoltre di salvare il file sorgente prima di provare ad eseguire script.

### assemble.ps1

```
PS /mnt/c/reti_logiche/assembler> ./assemble.ps1 mio_programma.s
```

Questo script assembla un sorgente assembler in un file eseguibile. Lo script controlla prima che il file passato non sia un eseguibile, invece che un sorgente. Poi, il sorgente viene assemblato usando gcc e includendo il sorgente ./files/main.c, che si occupa di alcune impostazioni del terminale.

### debug.ps1

```
PS /mnt/c/reti_logiche/assembler> ./debug.ps1 mio_programma
```

Questo script lancia il debugger per un programma. Lo script controlla prima che il file passato non sia un sorgente, invece che un eseguibile. Poi, il debugger gdb viene lanciato con il programma dato, includendo le definizioni e comandi iniziali in ./files/gdb\_startup. Questi si occupano di definire i comandi qquit e rrun (non chiedono conferma), creare un breakpoint in \_main e avviare il programma fino a tale breakpoint (così da saltare il codice di setup di ./files/main.c).

### run-test.ps1

```
PS /mnt/c/reti_logiche/assembler> ./run-test.ps1 mio_programma input.txt output.txt
```

Lancia un eseguibile usando il contenuto di un file come input, e opzionalmente ne stampa l'output su file. Lo script fa ridirezione di input/output, con alcuni controlli. Tutti i caratteri del file di input verranno visti dal programma come se digitati da tastiera, inclusi i caratteri di fine riga.

Nei computer dei laboratori, questo script si chiama attualmente test.ps1.

### run-tests.ps1

```
PS /mnt/c/reti_logiche/assembler> ./run-tests.ps1 mio_programma cartella_test
```

Testa un eseguibile su una serie di coppie input-output, verificando che l'output sia quello atteso. Stampa riassuntivamente e per ciascun test se è stato passato o meno.

Lo script prende ciascun file di input, con nome nella forma in\_\*.txt, ed esegue l'eseguibile con tale input. Ne salva poi l'output corrispondente nel file out\_\*.txt. Confronta poi out\_\*.txt e out\_ref\_\*.txt : il test è passato se i due file coincidono. Nel confronto, viene ignorata la differenza fra le sequenze di fine riga \r\n e \n.



## 7. Problemi comuni

Questa sezione include problemi che è frequente incontrare.

Come regola generale, in sede d'esame rispondiamo a tutte le domande relative a problemi di questo tipo e aiutiamo a proseguire - perché sono relative all'ambiente d'esame e non ai concetti *oggetto* d'esame.

Per altre domande, si può sempre contattare per email o Teams.

### 7.1 Setup dell'ambiente

#### 1. Ho trovato un ambiente assembler per Mac su Github, ma ho problemi a usarlo

Non abbiamo fatto noi quell'ambiente, non sappiamo come funziona e non offriamo supporto su come usarlo.

#### 2. Ho trovato un ambiente basato su DOS, usato precedentemente all'esame, ma ho problemi a usarlo

Ha probabilmente incontrato uno dei tanti motivi per cui l'ambiente basato su DOS è stato abbandonato. Questi problemi sono al più *aggirabili*, non *risolvibili*.

#### 3. Lanciando il file `assemble.code-workspace`, mi appare un messaggio del tipo **Unknown distro: Ubuntu**

Il file `assemble.code-workspace` cerca di lanciare via WSL la distribuzione chiamata Ubuntu, senza alcuna specifica di versione. Nel caso la vostra installazione sia diversa, andrà modificato il file. Da un terminale Windows, lanciare `wsl --list -v`, dovrete ottenere una stampa del tipo

```
PS C:\Users\raffa> wsl --list -v
NAME                STATE             VERSION
* Ubuntu            Stopped           2
Ubuntu-22.04        Stopped           2
```

La parte importante è la colonna NAME dell'immagine che vogliamo usare per l'ambiente assembler. Modificare il file `assemble.code-workspace` con un editor di testo (notepad o VS Code stesso, stando attenti ad aprirlo come file di testo e non come workspace) sostituendo tutte le occorrenze di `wsl+ubuntu` con `wsl+NOME-DELLA-DISTRO`. Per esempio, se volessi utilizzare l'immagine `Ubuntu-22.04`, sostituirei con `wsl+Ubuntu-22.04`.

#### 4. Sto utilizzando una sistema Linux desktop, come uso l'ambiente senza virtualizzazione?

Il file `assemble.code-workspace` fa tre cose

- Aprire VS Code nella macchina virtuale WSL
- Aprire la cartella `C:/reti_logiche/assembler` in tale ambiente
- Impostare `pwsh` come terminale default

È possibile fare manualmente gli step 2 e 3, o modificare `assemble.code-workspace` per non fare lo step 1. Per seguire questa seconda opzione, eliminare la riga con `"remoteAuthority":`, e modificare il percorso dopo `"uri":` perché sia semplicemente un percorso sul proprio disco, per esempio `"uri": "/home/raff/reti_logiche/assembler"`.

## 7.2 Uso dell'ambiente

### 5. Se premo **Run** su VS Code non viene lanciato il programma

Non è così che si usa l'ambiente di questo corso. Si deve usare un terminale, assemblare con `./assemble.ps1 programma.s` e lanciare con `./programma`.

### 6. Provando a lanciare `./assemble.ps1 programma.s` ricevo un errore del tipo `./assemble.ps1: line 1: syntax error near unexpected token`

State usando la shell da terminale sbagliata, bash invece che pwsh. Aprire un terminale Powershell da VS Code o utilizzare il comando `pwsh`.

### 7. Provando ad assemblare ricevo un warning del tipo `warning: creating DT_TEXTREL in a PIE`

Sostituire il file `assemble.ps1` con quello contenuto nel pacchetto più recente tra i file del corso. Oppure modificare manualmente il file, alla riga 29, da

```
gcc -m32 -o ...
```

a

```
gcc -m32 -no-pie -o ...
```

Riprovare quindi a riassemblare. Se il warning non sparisce, scrivermi. Allegando il sorgente.

### 8. Ho modificato il codice per correggere un errore, ma quando assemblo ed eseguo il codice, continuo a vedere lo stesso errore.

Controllare di aver salvato il file. In alto, nella barra delle tab, VS Code mostra un pallino pieno, al posto della X per chiedere la tab, per i file modificati e non salvati.

### 9. Dove trovo i file che scrivo nell'ambiente assembler?

La cartella `assembler` mostrata in VS Code corrisponde alla cartella `C:/reti_logiche/assembler` su Windows. Troveremo qui sia i file sorgenti (estensione `.s`) che i binari assemblati.



## 8. Essere efficienti con VS Code

VS Code è l'editor disponibile in sede d'esame e mostrato a lezione. Come ogni strumento di lavoro, è una buona idea imparare ad usarlo bene per essere più rapidi ed efficaci. Questo si traduce, in genere, nel prendere l'abitudine di usare meno il mouse e più la tastiera, usando le dovute scorciatoie e combinazioni di tasti.

In questa documentazione ci focalizziamo sulle combinazioni per Windows, che sono quelle che troverete all'esame. Evidenzierò con una ☆ le combinazioni più importanti e probabilmente meno note.

### Salvare i file

Fra le cause dei vari errori per cui riceviamo richieste d'aiuto, una delle più frequenti è che i file modificati non sono stati salvati. Un file modificato ma non salvato è indicato da un pallino nero nella tab in alto, e le modifiche non saranno visibili a altri programmi come gcc e iverilog.

Si consiglia di salvare spesso e abitualmente, usando `ctrl + s`.

### 8.1 Le basi elementari

Quando si scrive in un editor, il testo finisce dove sta il cursore (in inglese *caret*). È la barra verticale che indica dove stiamo scrivendo. Si può spostare usando le frecce, non solo destra e sinistra ma anche su e giù. Usando font monospace, infatti, il testo è una matrice di celle delle stesse dimensioni, ed è facile prevedere dove andrà il caret anche mentre ci si sposta tra le righe.

Vediamo quindi le combinazioni più comuni.

|   | Tasti                         | Cosa fa                                               |
|---|-------------------------------|-------------------------------------------------------|
|   | Tenere premuto shift          | Seleziona il testo seguendo il movimento del cursore. |
|   | <code>ctrl + c</code>         | Copia il testo selezionato.                           |
|   | <code>ctrl + v</code>         | Incolla il testo selezionato.                         |
|   | <code>ctrl + x</code>         | Taglia (cioè copia e cancella) il testo selezionato.  |
|   | <code>ctrl + f</code>         | Cerca all'interno del file.                           |
|   | <code>ctrl + h</code>         | Cerca e sostituisce all'interno del file.             |
| ☆ | <code>ctrl + s</code>         | Salva il file corrente.                               |
|   | <code>ctrl + shift + p</code> | Apre la <i>Command Palette</i> di VS Code.            |

### 8.2 Le basi un po' meno elementari

Si può spostare il cursore in modo ben più rapido che un carattere alla volta.

|   | Tasti                                                                                                                                   | Cosa fa                                                                               |
|---|-----------------------------------------------------------------------------------------------------------------------------------------|---------------------------------------------------------------------------------------|
| ☆ | <code>ctrl + freccia sx o dx</code>                                                                                                     | Sposta il cursore di un <i>token</i> (in genere una parola, ma dipende dal contesto). |
|   | <code>home</code> (inizio in italiano, più spesso  ) | Sposta il cursore all'inizio della riga.                                              |
|   | <code>end</code> (fine in italiano)                                                                                                     | Sposta il cursore alla fine della riga.                                               |
|   | <code>ctrl + shift + f</code>                                                                                                           | Cerca all'interno della cartella/progetto/...                                         |
|   | <code>ctrl + shift + h</code>                                                                                                           | Cerca e sostituisce all'interno della cartella/progetto/...                           |
|   | <code>alt + freccia su/giù</code>                                                                                                       | Sposta la riga corrente (o le righe selezionate) verso l'alto/basso.                  |
| ☆ | <code>ctrl + alt + freccia su/giù</code>                                                                                                | Copia la riga corrente (o le righe selezionate) verso l'alto/basso.                   |

### 8.3 Editing multi-caret

Normalmente c'è un cursore, e ogni modifica fatta viene applicata dov'è quel *singolo* cursore.

Negli esempi che seguono, userò | per indicare un cursore, e coppie di \_ come delimitatori del testo selezionato.

Contenu|to dell'editor

Premendo A

```
ContenuA|to dell'editor
```

L'idea del multi-caret è di avere più di un cursore, per modificare più punti del testo allo stesso tempo. Questo è utile se abbiamo più punti del testo con uno stesso *pattern*.

|   | Tasti    | Cosa fa                                                                        |
|---|----------|--------------------------------------------------------------------------------|
| ☆ | ctrl + d | Aggiunge un cursore alla fine della prossima occorrenza del testo selezionato. |
|   | esc      | Ritorno alla modalità con singolo cursore.                                     |

Vediamo un esempio.

```
Prima |riga dell'editor
Seconda riga dell'editor
Terza riga dell'editor
```

Si comincia selezionando del testo.

```
Prima _riga_| dell'editor
Seconda riga dell'editor
Terza riga dell'editor
```

Usiamo ora ctrl + d per mettere un nuovo caret dopo la prossima occorrenza di “riga”.

```
Prima _riga_| dell'editor
Seconda _riga_| dell'editor
Terza riga dell'editor
```

Abbiamo ora due caret e se facciamo una modifica verrà fatta in tutti e due i punti. Premendo per esempio e, andremo a sovrascrivere la parola “riga” in entrambi i punti.

```
Prima e| dell'editor
Seconda e| dell'editor
Terza riga dell'editor
```

Entrambi i cursori seguiranno indipendentemente anche gli altri comandi: movimento per caratteri, movimento per token, selezione, copia e incolla.

Per sfruttare questo, conviene scrivere codice secondo pattern in modo da facilitare questo tipo di modifiche. Per esempio, è utile avere cose che vorremmo poi modificare contemporaneamente su righe diverse, in modo da sfruttare home e end in modalità multi-cursore.

Vedremo in particolare come la sintesi di reti sincronizzate diventa molto più semplice se si sfrutta appieno l'editor.